

Module 17 Notes: Circular Singly Linked List Deletion

1. Current Progress Analysis

You are now at **Module 17**.

Before this module, you already learned the main ideas needed to understand deletion in a circular singly linked list.

Previous idea	Why it matters in Module 17
Node structure	You know that every node has <code>data</code> and <code>next</code> .
Pointer movement	You know that <code>p = p->next</code> moves a pointer to the next node.
Normal traversal	You know how to move through nodes one by one.
Normal singly deletion	You know that deletion means changing links and freeing memory.
Circular singly linked lists	You know that the last node points back to <code>Head</code> , not to <code>NULL</code> .
Circular insertion	You know that circular operations must preserve the circle.
Circular length/display	You know that circular traversal stops when the pointer returns to <code>Head</code> .

Up to Module 16, you learned how to **insert** into a circular singly linked list.

Module 17 teaches the opposite operation:

Instead of creating a node and connecting it, we disconnect a node and free it.

The main topic of this module is:

Deleting a node from a circular singly linked list without breaking the circle.

2. Circular Singly Linked List Reminder

A circular singly linked list is a linked list where the last node points back to the first node.

Normal singly linked list:

```
10 -> 20 -> 30 -> NULL
```

Circular singly linked list:

```
Head
|
v
10 -> 20 -> 30
^         |
|         |
```

The important rule is:

```
last->next == Head
```

In a normal singly linked list, the last node points to `NULL`.

In a circular singly linked list, the last node points back to `Head`.

So there is no natural `NULL` ending point.

3. Big Idea of Module 17

Deletion means removing a node from the list.

Suppose we have this circular singly linked list:

```
Head
|
v
8 -> 3 -> 9 -> 6 -> 2
^         |
|         |
```

If we delete `6`, the list should become:

```
Head
|
v
8 -> 3 -> 9 -> 2
```



The node **6** is removed because node **9** now points directly to node **2**.

The basic deletion idea is:

```
previous->next = target->next;
free(target);
```

Meaning:

1. Make the previous node skip the node we want to delete.
2. Release the deleted node from heap memory.

But circular deletion has one extra rule:

After deletion, the last node must still point back to **Head**.

That is the main difference between normal singly deletion and circular singly deletion.

4. Node Structure

A circular singly linked list uses the same node structure as a normal singly linked list.

```
struct Node {
    int data;
    struct Node *next;
};
```

Each node has two parts:

Part	Meaning
data	Stores the value inside the node.
next	Stores the address of the next node.

The node structure does not change.

Only the final link changes.

Normal final node:

```
[30 | NULL]
```

Circular final node:

```
[30 | address of Head]
```

5. Meaning of Head

In circular linked lists, the first pointer is usually called **Head**.

```
struct Node *Head = NULL;
```

Head points to the first node of the circular list.

Example:

```
Head
|
v
8 -> 3 -> 9 -> 6 -> 2
^                   |
|_____|
```

The list starts at **Head**.

But unlike a normal singly linked list, the list does not end at **NULL**.

So the rule is:

Start at **Head**, move through the list, and stop when you come back to **Head**.

6. Deletion Index Convention

Module 17 uses **1-based indexing** for deletion.

This means:

Index	Meaning
1	Delete the first node, which is <code>Head</code> .
2	Delete the second node.
3	Delete the third node.
n	Delete the last node in a list of length <code>n</code> .

Example list:

8 -> 3 -> 9 -> 6 -> 2 -> back to 8

This list has 5 nodes.

Valid deletion indexes are:

1, 2, 3, 4, 5

Invalid deletion indexes are:

0, -1, 6, 100

Important:

`Delete(0)` is invalid in Module 17.

This is different from insertion.

In Module 16 insertion, index `0` meant insert before `Head`.

But in Module 17 deletion, index `0` is not allowed.

7. Why Index Validation Is Needed

Before deleting a node, we must check whether the index is valid.

The rule is:

```
if (index < 1 || index > Length(Head)) {
    return -1;
}
```

Meaning:

Condition	Meaning
<code>index < 1</code>	The index is too small. Deletion starts at index <code>1</code> .
<code>index > Length(Head)</code>	The index is larger than the number of nodes.

If the index is invalid, the function returns `-1`.

The list should not be changed.

Example:

8 -> 3 -> 9 -> back to 8

Length is `3`.

Valid deletion indexes:

1, 2, 3

Invalid deletion indexes:

0, 4, 5, 100

If we do not validate the index, pointer movement may go to the wrong place and damage the list.

8. Circular Length Function

To validate the index, we need to know how many nodes are in the circular list.

In a normal singly linked list, we often count like this:

```
while (p != NULL)
```

But this is wrong for a circular linked list.

Why?

Because a circular linked list may never reach `NULL`.

Example:

```
8 -> 3 -> 9
^         |
|         |
|         |
```

If we write:

```
while (p != NULL)
```

then `p` may keep moving forever.

So circular length must stop when `p` returns to `Head`.

9. Circular Length Code

```
int Length(struct Node *h) {
    if (h == NULL) {
        return 0;
    }

    int len = 0;
    struct Node *p = h;

    do {
        len++;
        p = p->next;
    } while (p != h);

    return len;
}
```

10. Explanation of the Length Function

Function header

```
int Length(struct Node *h)
```

This function receives a pointer to the first node of the circular list.

It returns the number of nodes.

Empty list check

```
if (h == NULL) {  
    return 0;  
}
```

If `Head` is `NULL`, there are no nodes.

So the length is `0`.

Create a counter and pointer

```
int len = 0;  
struct Node *p = h;
```

`len` counts the nodes.

`p` moves through the list.

We use `p` instead of moving `Head` directly.

Count using `do-while`

```
do {  
    len++;  
    p = p->next;  
} while (p != h);
```


This means:

1. Count the current node.
2. Move to the next node.
3. Stop when we return to the starting node.

11. Why `do-while` Is Used

At the beginning:

```
p == h
```

If we used:

```
while (p != h)
```

then the loop would not run even once.

That would incorrectly return `0` for a non-empty list.

So we use `do-while` because it runs the body first and checks the condition after.

Simple rule:

In a circular list, visit the first node first, then move, then check whether you returned to `Head`.

12. Dry Run of Length

List:

```
Head
|
v
8 -> 3 -> 9 -> 6 -> 2
^               |
|_____|
```

Call:

```
Length(Head)
```

Start:

```
p = Head, so p points to 8  
len = 0
```

Dry run:

Step	p points to	Action	len
1	8	Count 8, move to 3	1
2	3	Count 3, move to 9	2
3	9	Count 9, move to 6	3
4	6	Count 6, move to 2	4
5	2	Count 2, move back to 8	5

Now `p == Head`, so the loop stops.

Final result:

```
Length = 5
```

13. Main Deletion Cases

Circular singly linked list deletion has these main cases:

1. Delete from an empty list.
2. Delete the only node.
3. Delete `Head` from a multi-node list.
4. Delete a middle node.
5. Delete the last node.

The code usually groups them like this:

Case	What happens
Empty list	Return <code>-1</code> .

Case	What happens
Invalid index	Return <code>-1</code> .
Delete index <code>1</code>	Delete <code>Head</code> .
Delete any other index	Delete middle or last node.

Case 1: Delete From an Empty List

14. Empty List Deletion Idea

An empty circular singly linked list has no nodes.

```
Head == NULL
```

Picture:

```
Head -> NULL
```

If we try to delete from an empty list, there is nothing to delete.

Example:

```
Delete(1);
```

This should fail.

So we return:

```
-1
```

15. Empty List Code Pattern

```
if (Head == NULL) {
    return -1;
}
```

Meaning:

If the list is empty, do not continue.

Return `-1` to show that deletion failed.

Case 2: Delete the Only Node

16. One-Node Circular List

A one-node circular list looks like this:

```
Head
|
v
10
^|
||
|_|
```

Here:

```
Head->next == Head
```

The node points to itself because it is both the first node and the last node.

If we delete this node, the list becomes empty:

```
Head = NULL;
```

17. One-Node Deletion Steps

Suppose the list is:

```
10 -> back to 10
```

Call:

```
Delete(1);
```

Steps:

1. Save the data.
2. Free the node.
3. Set `Head` to `NULL`.
4. Return the deleted value.

Code idea:

```
x = Head->data;  
free(Head);  
Head = NULL;  
return x;
```

Important:

This case must be handled carefully so that `Head` does not point to freed memory.

Case 3: Delete `Head` From a Multi-Node Circular List

18. Head Deletion Idea

Suppose the list is:

```
Head  
|  
v  
8 -> 3 -> 9  
^       |  
|_____|
```

Call:

```
Delete(1);
```

We want to delete `8`, because index `1` means delete `Head`.

Final result:

```
Head
|
v
3 -> 9
^   |
|___|
```

The new `Head` should be node `3`.

But there is a circular-list problem.

Before deletion:

```
9->next == Head
```

This means node `9` points to node `8`.

After deleting node `8`, node `9` must point to node `3`.

So we must update the last node.

19. Why We Must Find the Last Node

In a circular singly linked list, each node only points forward.

From `Head`, we can move like this:

```
8 -> 3 -> 9 -> back to 8
```

But we cannot move backward directly from `Head` to the last node.

So to update the last node, we must search for it.

The last node is the node whose `next` points to `Head`.

Condition:

```
p->next == Head
```

To find the last node:

```
p = Head;
while (p->next != Head) {
    p = p->next;
}
```

When the loop stops, `p` points to the last node.

20. Head Deletion Link Changes

After finding the last node, use this idea:

```
q = Head;
x = q->data;
p->next = Head->next;
Head = Head->next;
free(q);
return x;
```

Meaning:

Line	Meaning
<code>q = Head;</code>	Save the old <code>Head</code> node so we can free it later.
<code>x = q->data;</code>	Save the deleted value.
<code>p->next = Head->next;</code>	Make the last node point to the new <code>Head</code> .
<code>Head = Head->next;</code>	Move <code>Head</code> to the second node.
<code>free(q);</code>	Delete the old <code>Head</code> node.
<code>return x;</code>	Return the deleted value.

21. Why the Order Matters in Head Deletion

Correct order:

```
q = Head;
x = q->data;
p->next = Head->next;
Head = Head->next;
free(q);
```

This is safe because:

1. We save the old `Head` in `q`.
2. We save the data before freeing the node.
3. We make the last node point to the new `Head`.
4. We move `Head` forward.
5. We free the old `Head`.

Wrong idea:

```
free(Head);
Head = Head->next;
```

This is unsafe.

After `free(Head)`, we should not use `Head->next`.

The node has already been released.

22. Dry Run: Delete `Head`

List:

```
Head
|
v
8 -> 3 -> 9
^         |
|         |
```

Call:

```
Delete(1);
```

First, find the last node:


```
p starts at 8
p moves to 3
p moves to 9
p->next == Head, so stop
```

Now:

```
p points to 9
Head points to 8
```

Do:

```
q = Head;           // q points to 8
x = q->data;         // x = 8
p->next = Head->next; // 9 points to 3
Head = Head->next;   // Head moves to 3
free(q);             // delete 8
```

Final list:

```
Head
|
v
3 -> 9
^   |
|___|
```

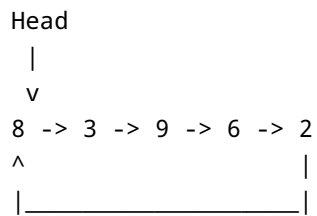
Returned value:

```
8
```

Case 4: Delete a Middle Node

23. Middle Deletion Idea

Suppose the list is:



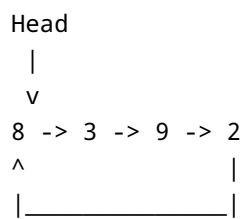
Call:

```
Delete(4);
```

Index **4** means delete the fourth node.

The fourth node is **6**.

Final result:



The node before **6** is **9**.

So we must make **9** point directly to **2**.

24. Pointers Used for Middle Deletion

Middle deletion uses two pointers:

Pointer	Meaning
p	Points to the node before the node we want to delete.
q	Points to the node we want to delete.

For deleting index **4**:

```
8 -> 3 -> 9 -> 6 -> 2
      p   q
```

Here:

```
p points to 9
q points to 6
```

The goal is:

```
p->next = q->next;
```

This makes 9 point to 2.

25. How to Move p to the Correct Place

For deleting an index greater than 1, p must stop at the node before the target.

The loop is:

```
p = Head;
for (int i = 0; i < index - 2; i++) {
    p = p->next;
}
```

Why `index - 2`?

Because:

- p starts at the first node.
- We want p to stop one node before the node being deleted.

Example:

List:

```
8 -> 3 -> 9 -> 6 -> 2 -> back to 8
```

Delete index 4.

We want `p` to stop at node `9`.

Movement:

Movement	<code>p</code> points to
Start	8
1	3
2	9

So `p` moves `2` times.

And:

```
index - 2 = 4 - 2 = 2
```

26. Middle Deletion Link Changes

After `p` reaches the node before the target:

```
q = p->next;
x = q->data;
p->next = q->next;
free(q);
return x;
```

Meaning:

Line	Meaning
<code>q = p->next;</code>	<code>q</code> points to the node we want to delete.
<code>x = q->data;</code>	Save the deleted value.
<code>p->next = q->next;</code>	Make <code>p</code> skip <code>q</code> .
<code>free(q);</code>	Release the deleted node.
<code>return x;</code>	Return the deleted value.

27. Dry Run: Delete Middle Node

List:

```
8 -> 3 -> 9 -> 6 -> 2 -> back to 8
```

Call:

```
Delete(4);
```

We want to delete the fourth node: 6.

Start:

```
p = Head = 8
```

Move `p` `index - 2` times:

```
index - 2 = 2
```

Movement	<code>p</code> points to
Start	8
1	3
2	9

Now:

```
8 -> 3 -> 9 -> 6 -> 2
      p   q
```

Do:

```
q = p->next;    // q points to 6
x = q->data;     // x = 6
p->next = q->next; // 9 now points to 2
free(q);        // delete 6
```

Final list:

```
8 -> 3 -> 9 -> 2 -> back to 8
```

Returned value:

```
6
```

Case 5: Delete the Last Node

28. Last Node Deletion Idea

Deleting the last node uses the same logic as deleting a middle node.

Example list:

```
Head
|
v
8 -> 3 -> 9 -> 6 -> 2
^                   |
|_____|
```

Call:

```
Delete(5);
```

Index **5** means delete node **2**, the last node.

Before deletion:

```
6 -> 2 -> back to 8
p    q
```

After deletion:

```
6 -> back to 8
```

The previous node `6` becomes the new last node.

So after deletion:

```
6->next == Head
```

29. Why the Same Code Works for Last Deletion

For last deletion:

```
q = p->next;  
p->next = q->next;
```

If `q` is the last node, then:

```
q->next == Head
```

So this line:

```
p->next = q->next;
```

becomes:

```
p->next = Head;
```

That means the previous node becomes the new last node and points back to `Head`.

So the same middle-deletion code also works for deleting the last node.

30. Dry Run: Delete Last Node

List:

8 -> 3 -> 9 -> 6 -> 2 -> back to 8

Call:

Delete(5);

We want to delete node 2.

Move p index - 2 times:

index - 2 = 3

Movement	p points to
Start	8
1	3
2	9
3	6

Now:

8 -> 3 -> 9 -> 6 -> 2
 p q

Do:

```
q = p->next;      // q points to 2
x = q->data;       // x = 2
p->next = q->next; // 6 points back to Head, which is 8
free(q);          // delete 2
```

Final list:

8 -> 3 -> 9 -> 6 -> back to 8

Returned value:

31. Complete C Code for Module 17

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *Head = NULL;

int Length(struct Node *h) {
    if (h == NULL) {
        return 0;
    }

    int len = 0;
    struct Node *p = h;

    do {
        len++;
        p = p->next;
    } while (p != h);

    return len;
}

int Delete(int index) {
    struct Node *p;
    struct Node *q;
    int x;

    if (Head == NULL) {
        return -1;
    }

    if (index < 1 || index > Length(Head)) {
        return -1;
    }
}
```

```

// Case 1: Delete Head
if (index == 1) {
    p = Head;

    // Find the last node
    while (p->next != Head) {
        p = p->next;
    }

    x = Head->data;

    // One-node circular list
    if (p == Head) {
        free(Head);
        Head = NULL;
    }
    // Multi-node circular list
    else {
        q = Head;
        p->next = Head->next;
        Head = Head->next;
        free(q);
    }

    return x;
}

// Case 2: Delete middle or last node
p = Head;

for (int i = 0; i < index - 2; i++) {
    p = p->next;
}

q = p->next;
x = q->data;
p->next = q->next;
free(q);

return x;
}

```

32. Explanation of the Complete Delete Function

Function header

```
int Delete(int index)
```

This function deletes the node at the given index.

It returns:

Return value	Meaning
Deleted value	Deletion succeeded.
-1	Deletion failed.

Local variables

```
struct Node *p;  
struct Node *q;  
int x;
```

Variable	Meaning
p	Used to move through the list.
q	Used to hold the node being deleted.
x	Stores the deleted value before freeing the node.

Empty list check

```
if (Head == NULL) {  
    return -1;  
}
```

If the list is empty, there is no node to delete.

Index validation

```
if (index < 1 || index > Length(Head)) {  
    return -1;  
}
```

If the index is outside the valid range, deletion fails.

Delete Head

```
if (index == 1) {  
    ...  
}
```

Index 1 means delete the first node.

This case is special because deleting Head changes the starting point of the circular list.

Find the last node

```
p = Head;  
while (p->next != Head) {  
    p = p->next;  
}
```

This moves p to the last node.

We need the last node because its next must be updated when Head changes.

Save deleted value

```
x = Head->data;
```

We save the value before freeing the node.

After a node is freed, we should not access its data.

One-node case

```
if (p == Head) {
    free(Head);
    Head = NULL;
}
```

If `p == Head`, the list has only one node.

After deleting it, the list becomes empty.

Multi-node Head deletion

```
else {
    q = Head;
    p->next = Head->next;
    Head = Head->next;
    free(q);
}
```

This deletes the old `Head` and moves `Head` to the next node.

The last node now points to the new `Head`.

Middle or last deletion

```
p = Head;

for (int i = 0; i < index - 2; i++) {
    p = p->next;
}

q = p->next;
x = q->data;
p->next = q->next;
free(q);
```

This deletes any node except the first node.

`p` stops before the target node.

`q` points to the target node.

Then `p` skips `q`.

33. Full Example Program for Testing

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *Head = NULL;

void Create(int A[], int n) {
    if (n <= 0) {
        Head = NULL;
        return;
    }

    struct Node *t;
    struct Node *last;

    Head = (struct Node *)malloc(sizeof(struct Node));
    Head->data = A[0];
    Head->next = Head;
    last = Head;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = Head;
        last->next = t;
        last = t;
    }
}

void Display(struct Node *h) {
    if (h == NULL) {
        printf("List is empty\n");
        return;
    }
}
```

```

    struct Node *p = h;

    do {
        printf("%d ", p->data);
        p = p->next;
    } while (p != h);

    printf("\n");
}

int Length(struct Node *h) {
    if (h == NULL) {
        return 0;
    }

    int len = 0;
    struct Node *p = h;

    do {
        len++;
        p = p->next;
    } while (p != h);

    return len;
}

int Delete(int index) {
    struct Node *p;
    struct Node *q;
    int x;

    if (Head == NULL) {
        return -1;
    }

    if (index < 1 || index > Length(Head)) {
        return -1;
    }

    if (index == 1) {
        p = Head;

        while (p->next != Head) {
            p = p->next;
        }

        x = Head->data;

```

```

        if (p == Head) {
            free(Head);
            Head = NULL;
        } else {
            q = Head;
            p->next = Head->next;
            Head = Head->next;
            free(q);
        }

        return x;
    }

    p = Head;

    for (int i = 0; i < index - 2; i++) {
        p = p->next;
    }

    q = p->next;
    x = q->data;
    p->next = q->next;
    free(q);

    return x;
}

int main() {
    int A[] = {8, 3, 9, 6, 2};

    Create(A, 5);

    printf("Original list: ");
    Display(Head);

    printf("Deleted value: %d\n", Delete(4));
    printf("After deleting index 4: ");
    Display(Head);

    printf("Deleted value: %d\n", Delete(1));
    printf("After deleting Head: ");
    Display(Head);

    return 0;
}

```


Expected output:

```
Original list: 8 3 9 6 2
Deleted value: 6
After deleting index 4: 8 3 9 2
Deleted value: 8
After deleting Head: 3 9 2
```

34. Complexity Analysis

Let the list have `n` nodes.

Operation	Time Complexity	Why
Delete from empty list	<code>O(1)</code>	Only checks <code>Head == NULL</code> .
Invalid index check	<code>O(n)</code>	Calls <code>Length(Head)</code> .
Delete the only node	<code>O(1)</code> after validation	Frees one node and sets <code>Head = NULL</code> .
Delete <code>Head</code> from multi-node list	<code>O(n)</code>	Must find the last node first.
Delete middle node	<code>O(n)</code>	May need to move through the list.
Delete last node	<code>O(n)</code>	Must reach the node before the last node.
Extra space	<code>O(1)</code>	Only a few pointer variables are used.

Important note:

If the function always calls `Length(Head)` for validation, then the validation itself costs `O(n)`.

So even if the actual deletion is quick, the full function may still take `O(n)` because of validation.

35. Comparison With Normal Singly Linked List Deletion

Topic	Normal singly linked list	Circular singly linked list
Last node points to	NULL	Head
Traversal stops at	NULL	Returning to Head
Delete first node	Move first to first->next	Must also update last node's next
Delete middle node	p->next = q->next	Same idea
Delete last node	Previous node points to NULL	Previous node points to Head
Risk	Losing nodes	Breaking the circle

The biggest difference is deleting Head .

In a normal singly linked list, deleting the first node is simple:

```
first = first->next;
```

In a circular singly linked list, deleting Head needs extra work because the last node must point to the new Head .

36. Common Beginner Mistakes

Mistake 1: Using index 0 for deletion

Wrong:

```
Delete(0);
```

Correct:

```
Delete(1); // delete Head
```

Deletion uses indexes 1 to n .

Mistake 2: Using `while (p != NULL)`

Wrong for circular lists:

```
while (p != NULL)
```

A circular list may never reach `NULL`.

Correct when finding the last node:

```
while (p->next != Head)
```

Mistake 3: Freeing before saving the deleted value

Wrong:

```
free(q);  
x = q->data;
```

Correct:

```
x = q->data;  
free(q);
```

After `free(q)`, the node should not be used.

Mistake 4: Deleting `Head` without updating the last node

Wrong idea:

```
Head = Head->next;  
free(oldHead);
```

This is incomplete because the old last node may still point to the deleted node.

Correct idea:

```
last->next = Head->next;  
Head = Head->next;  
free(oldHead);
```

Mistake 5: Forgetting the one-node case

Wrong idea:

```
Head = Head->next;
```

In a one-node circular list:

```
Head->next == Head
```

So this would make `Head` point to itself even after deletion.

Correct idea:

```
free(Head);  
Head = NULL;
```

Mistake 6: Accessing a pointer after freeing it

Wrong:

```
free(q);  
printf("%d", q->data);
```

Correct:

```
x = q->data;  
free(q);  
printf("%d", x);
```

Once memory is freed, do not access that node again.

37. Key Pointer Patterns to Memorize

Find the last node

```
p = Head;
while (p->next != Head) {
    p = p->next;
}
```

Use this when you need the last node of a circular singly linked list.

Delete the only node

```
x = Head->data;
free(Head);
Head = NULL;
return x;
```

Use this when the circular list has exactly one node.

Delete **Head** from a multi-node list

```
q = Head;
x = q->data;
p->next = Head->next;
Head = Head->next;
free(q);
return x;
```

Use this when deleting index **1** from a list with more than one node.

Delete middle or last node

```
q = p->next;
x = q->data;
p->next = q->next;
```

```
free(q);  
return x;
```

Use this when deleting any node except `Head`.

38. Module 17 Summary

Module 17 teaches how to delete nodes from a circular singly linked list.

The main ideas are:

1. A circular singly linked list does not end at `NULL`.
 2. The last node points back to `Head`.
 3. Deletion indexes start from `1`, not `0`.
 4. `Delete(1)` deletes `Head`.
 5. Deleting `Head` is special because the last node must point to the new `Head`.
 6. A one-node circular list must become empty after deletion.
 7. Middle and last deletion use the normal skip-over idea.
 8. Always save the deleted value before freeing the node.
 9. Never use `while (p != NULL)` for circular traversal.
 10. Always preserve the circular link after deletion.
-

39. One-Line Memory Rule

Circular deletion = normal deletion + preserve the link back to Head.

This is the main lesson of Module 17.

If the circle remains valid after deletion, the operation is correct.

If the last node no longer points back to `Head`, the circular linked list is broken.

40. Practice Questions

Concept Questions

1. Why is `Delete(0)` invalid in Module 17?
2. Why can we not use `while (p != NULL)` in a circular singly linked list?

3. Why do we need to find the last node before deleting `Head`?
 4. What does `Head->next == Head` mean?
 5. What should happen after deleting the only node?
 6. Why must we save `x = q->data` before calling `free(q)`?
 7. Why does deleting the last node work using the same code as middle deletion?
-

Code Trace Practice

Given this list:

```
5 -> 10 -> 15 -> 20 -> back to 5
```

Trace these calls:

```
Delete(1);  
Delete(2);  
Delete(3);  
Delete(4);
```

For each call, answer:

1. Which node is deleted?
 2. What value is returned?
 3. What does the final list look like?
 4. Which pointer changes are made?
-

Mastery Check

You understand Module 17 when you can explain this without looking:

```
q = p->next;  
x = q->data;  
p->next = q->next;  
free(q);
```

In simple words:

1. `q` points to the node we want to delete.
2. `x` saves the deleted value.
3. `p->next = q->next` removes `q` from the chain.

4. `free(q)` releases the memory.

That is the core of linked-list deletion.

For circular deletion, the extra rule is:

The list must still come back to Head.